

如何在 Cloud Run worker 集區中代管單輪 ADK 代理

來源：<https://codelabs.developers.google.com/codelabs/cloud-run/host-adk-agent-worker-pool-single-turn?hl=zh-tw>

步驟數：8

瞭解如何將 ADK 代理程式做為工作站集區，用於單輪對話

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 設定和需求](#)
4. [4. 建立 Cloud Run worker 集區](#)
5. [5. 建構與部署](#)
6. [6. 測試代理](#)
7. [7. 恭喜！](#)
8. [8. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

本程式碼實驗室說明如何使用 Agent Development Kit (ADK) 建構可擴充的非同步代理系統。您將建立 Cloud Run 工作站集區，用於代管 ADK 快速入門天氣代理程式，處理 Pub/Sub 提取訂閱項目中的工作。

課程內容

- 如何使用 Agent Development Kit (ADK) 建立單輪代理。
 - 如何部署從 Pub/Sub 訂閱項目提取資料的 Cloud Run worker 集區。
-

步驟 2 · 約 0 分鐘

2. 事前準備

啟用 API

開始使用本程式碼研究室前，請先執行下列指令，啟用下列 API：

```
gcloud services enable \  
  run.googleapis.com \  
  cloudbuild.googleapis.com \  
  artifactregistry.googleapis.com \  
  pubsub.googleapis.com \  
  aiplatform.googleapis.com
```

步驟 3 · 約 0 分鐘

3. 設定和需求

如要設定必要資源，請按照下列步驟操作：

1. 為本程式碼研究室設定環境變數：



```
export PROJECT_ID=<YOUR_PROJECT_ID>
export REGION=europe-west4

# AR repo
export AR_REPO="codelab-agent-wp"

# Application Names
export WORKER_APP_NAME="multi-tool-agent-worker"

# Pub/Sub Resources
export MY_TOPIC="pull-pubsub-topic-agent"
export MY_SUBSCRIPTION="agent-wp-sub"

# Service Accounts
export WORKER_SA_NAME="agent-worker-sa"
export WORKER_SA_ADDRESS="${WORKER_SA_NAME}@${PROJECT_ID}.iam.gserviceaccount.com"
```

建立服務帳戶

為確保安全，我們會為工作人員建立專屬服務帳戶，確保該帳戶只具備所需權限。

為工作人員建立服務帳戶：

```
gcloud iam service-accounts create ${WORKER_SA_NAME} \
  --display-name="Service Account for ADK Agent Worker"
```

將必要角色授予服務帳戶。這項服務必須從 Pub/Sub 提取訊息，並叫用 ADK 使用的 Vertex AI 模型。

```
# Role for subscribing to Pub/Sub
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${WORKER_SA_ADDRESS}" \
  --role="roles/pubsub.admin"

# Role for invoking Vertex AI
gcloud projects add-iam-policy-binding ${PROJECT_ID} \
  --member="serviceAccount:${WORKER_SA_ADDRESS}" \
  --role="roles/aiplatform.user"
```

建立 Pub/Sub 資源

建立 Pub/Sub 主題，做為工作佇列。

```
gcloud pubsub topics create $MY_TOPIC
```

為工作站建立 Pub/Sub 訂閱項目，以便從中提取訊息。

```
gcloud pubsub subscriptions create $MY_SUBSCRIPTION --topic=$MY_TOPIC
```

步驟 4 · 約 0 分鐘

4. 建立 Cloud Run worker 集區

建立名為 `agents-wp` 的專案目錄。

```
mkdir agents-wp && cd agents-wp
```

建立 `Dockerfile`

```
touch Dockerfile
```

並在 `Dockerfile` 中新增下列內容

```
FROM python:3.11-slim
WORKDIR /app

# Create a non-root user
RUN adduser --disabled-password --gecos "" myuser

# Switch to the non-root user
USER myuser

# Set up environment variables
ENV PATH="/home/myuser/.local/bin:$PATH"

# Copy agent files
COPY --chown=myuser:myuser multi_tool_agent/ /app/multi_tool_agent/

# Install dependencies from requirements.txt
RUN pip install -r /app/multi_tool_agent/requirements.txt

# Set the entrypoint to run the agent as a worker
CMD ["python3", "/app/multi_tool_agent/main.py"]
```

在其中建立名為 `multi_tool_agent` 的子目錄。請注意資料夾名稱 `multi_tool_agent` 中的底線。這個資料夾的名稱必須與您稍後部署的 ADK 代理名稱相符。

```
mkdir multi_tool_agent && cd multi_tool_agent
```

建立 `__init__.py` 檔案

```
touch __init__.py
```

並將下列內容新增至 `__init__.py` 檔案

```
from . import agent
```

建立 `agent.py` 檔案

```
touch agent.py
```

並在 `agent.py` 檔案中新增下列內容


```

import datetime
from zoneinfo import ZoneInfo

from google.adk.agents.llm_agent import Agent

def get_weather(city: str) -> dict:
    """Retrieves the current weather report for a specified city.

    Args:
        city (str): The name of the city for which to retrieve the weather report.

    Returns:
        dict: status and result or error msg.
    """
    print(f"--- Entering get_weather function for city: {city} ---")
    if city.lower() == "new york":
        result = {
            "status": "success",
            "report": (
                "The weather in New York is sunny with a temperature of 25 degrees"
                " Celsius (77 degrees Fahrenheit).",
            ),
        }
    else:
        result = {
            "status": "error",
            "error_message": f"Weather information for '{city}' is not available.",
        }
    print(f"--- Exiting get_weather function with result: {result} ---")
    return result

def get_current_time(city: str) -> dict:
    """Returns the current time in a specified city.

    Args:
        city (str): The name of the city for which to retrieve the current time.

    Returns:
        dict: status and result or error msg.
    """
    print(f"--- Entering get_current_time function for city: {city} ---")
    if city.lower() == "new york":
        tz_identifier = "America/New_York"
    else:
        result = {
            "status": "error",
            "error_message": (
                f"Sorry, I don't have timezone information for {city}."
            ),
        }
    print(f"--- Exiting get_current_time function with result: {result} ---")

```

```

        return result

    tz = ZoneInfo(tz_identifier)
    now = datetime.datetime.now(tz)
    report = (
        f'The current time in {city} is {now.strftime("%Y-%m-%d %H:%M:%S %Z%z")}'
    )
    result = {"status": "success", "report": report}
    print(f"--- Exiting get_current_time function with result: {result} ---")
    return result

print("--- Creating root_agent ---")
root_agent = Agent(
    name="weather_time_agent",
    model="gemini-2.5-flash",
    description=(
        "Agent to answer questions about the time and weather in a city."
    ),
    instruction=(
        "You are a helpful agent who can answer user questions about the time and weather in a city."
    ),
    tools=[get_weather, get_current_time],
)
print("--- root_agent created ---")

```

建立 `main.py` 檔案

```
touch main.py
```

並將下列內容新增至 `main.py` 檔案

```

import asyncio
import os

from google.adk.runners import InMemoryRunner, Runner
from google.genai import types
from google.cloud import pubsub_v1

from agent import root_agent

# --- Runner-based Invocation with Proper Async Handling ---

APP_NAME = "multi_tool_agent_worker"
USER_ID = "pubsub_user"

async def process_message(runner: Runner, message_data: bytes):
    """Processes a single message using the agent runner."""
    print(f"Processing message: {message_data}")
    try:
        prompt = message_data.decode("utf-8")
        session = await runner.session_service.create_session(
            app_name=APP_NAME,
            user_id=USER_ID
        )
        final_response_text = ""
        async for event in runner.run_async(
            user_id=USER_ID,
            session_id=session.id,
            new_message=types.Content(
                role="user", parts=[types.Part.from_text(text=prompt)]
            ),
        ):
            if event.content and event.content.parts:
                if event.author != "user":
                    # Filter out thought parts to get only the final response text
                    final_response_text += "".join(
                        part.text or "" for part in event.content.parts if not part.thought
                    )
        print(f"Agent response: {final_response_text}")

    except Exception as e:
        print(f"Error processing message: {e}")

async def async_worker(queue: asyncio.Queue, runner: Runner):
    """Continuously gets messages from the queue and processes them."""
    while True:
        message = await queue.get()
        if message is None: # Sentinel for stopping
            break
        await process_message(runner, message.data)
        message.ack()
        queue.task_done()

```

```

async def main():
    """Sets up the Pub/Sub subscriber and the async worker."""
    project_id = os.environ.get("GOOGLE_CLOUD_PROJECT")
    subscription_id = os.environ.get("SUBSCRIPTION_ID")

    if not project_id or not subscription_id:
        print("GOOGLE_CLOUD_PROJECT and SUBSCRIPTION_ID environment variables must be set.")
        return

    runner = InMemoryRunner(agent=root_agent, app_name=APP_NAME)
    message_queue = asyncio.Queue()

    subscriber = pubsub_v1.SubscriberClient()
    subscription_path = subscriber.subscription_path(project_id, subscription_id)

    loop = asyncio.get_running_loop()

    callback = lambda message: loop.call_soon_threadsafe(
        message_queue.put_nowait, message
    )

    print(f"Listening for messages on {subscription_path}...\n")
    streaming_pull_future = subscriber.subscribe(subscription_path, callback=callback)

    worker_task = asyncio.create_task(async_worker(message_queue, runner))

    try:
        # This will block until the subscription is cancelled or an error occurs.
        await loop.run_in_executor(None, streaming_pull_future.result)
    except KeyboardInterrupt:
        print("Shutting down...")
    finally:
        streaming_pull_future.cancel()
        await message_queue.put(None) # Stop the worker
        await worker_task # Wait for the worker to finish
        await runner.close()
        subscriber.close()

if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        print("Exiting.")

```

建立 requirements.txt 檔案

```
touch requirements.txt
```

並在 requirements.txt 檔案中加入以下內容

```
google-adk  
google-cloud-pubsub  
google-cloud-aiplatform
```

資料夾結構應如下所示

```
agents-wp  
- multi_tool_agent  
  - __init__.py  
  - agent.py  
  - main.py  
  - requirements.txt  
- Dockerfile
```

步驟 5 · 約 0 分鐘

5. 建構與部署

建立 Artifact Registry 存放區

您需要儲存容器映像檔的位置。

```
gcloud artifacts repositories create codelab-agent-wp \
  --repository-format=docker \
  --location=${REGION} \
  --description="Repo for Cloud Run source deployments"
```

建構容器映像檔

前往 Dockerfile 所在的根目錄 agents-wp

```
cd ..
```

並執行下列建構指令。

```
gcloud builds submit . --tag \
  ${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO}/${WORKER_APP_NAME}:latest
```

部署至 Cloud Run

部署代理工作站映像檔。

```
gcloud beta run worker-pools deploy ${WORKER_APP_NAME} \
  --image=${REGION}-docker.pkg.dev/${PROJECT_ID}/${AR_REPO}/${WORKER_APP_NAME}:latest \
  --service-account=${WORKER_SA_ADDRESS} \
  --region=${REGION} \
  --set-env-vars="SUBSCRIPTION_ID=${MY_SUBSCRIPTION}" \
  --set-env-vars="PYTHONUNBUFFERED=1" \
  --set-env-vars="GOOGLE_GENAI_USE_VERTEXAI=1" \
  --set-env-vars="GOOGLE_CLOUD_PROJECT=${PROJECT_ID}" \
  --set-env-vars="GOOGLE_CLOUD_LOCATION=${REGION}"
```

步驟 6 · 約 0 分鐘

6. 測試代理

您可以將訊息直接發布至 Pub/Sub 主題，藉此測試工作站。

```
gcloud pubsub topics publish ${MY_TOPIC} --message="What is the weather in New York?"
```

您可以執行這項指令，在 Google Cloud 控制台中查看 multi-tool-agent-worker 服務的記錄。

```
gcloud logging read 'resource.type="cloud_run_worker_pool" AND  
resource.labels.worker_pool_name="'$WORKER_APP_NAME'" AND  
resource.labels.location="'$REGION'"' --limit 10 --format="value(textPayload)"
```

您應該會看到輸出內容，指出系統已收到並處理訊息，接著是代理程式的回應。

```
Agent response: The weather in New York is sunny with a temperature of 25 degrees Celsius (77  
degrees Fahrenheit).
```

步驟 7 · 約 0 分鐘

7. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Cloud Run](#) 說明文件，瞭解[工作站集區](#)和[主機代理程式](#)。

涵蓋內容

- 如何使用 Agent Development Kit (ADK) 建立單輪代理。
 - 如何部署從 Pub/Sub 訂閱項目提取資料的 Cloud Run worker 集區。
-

步驟 8 · 約 0 分鐘

8. 清除所用資源

為避免產生任何費用，請刪除您建立的資源。

刪除 Cloud Run worker 集區

```
gcloud beta run worker-pools delete ${WORKER_APP_NAME} --region=${REGION}
```

刪除 Pub/Sub 資源

```
gcloud pubsub subscriptions delete ${MY_SUBSCRIPTION}
```

```
gcloud pubsub topics delete ${MY_TOPIC}
```

刪除 Artifact Registry 存放區

```
gcloud artifacts repositories delete ${AR_REPO} --location=$REGION
```

刪除服務帳戶

```
gcloud iam service-accounts delete ${WORKER_SA_ADDRESS}
```

如要刪除整個專案，請前往「管理資源」，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
